

The SFM Data Pipeline Project

One project, three lanes: Analyst → Engineer → Cloud/SysAdmin. Recreates your real Bureau reporting workflow so every phase doubles as interview prep.

Why this project works

This rebuilds the exact monthly reporting you did at the NYC Bureau of Chronic Disease Prevention (Stellar Farmers Markets / PEARS), using mock data shaped like the real export: 13 sites, 132 sessions, 1,572 participants across Jul–Sep. When you finish, you can narrate your Bureau job cold in any interview — because you'll have just done the whole workflow with your own hands — AND you'll have a portfolio piece that shows Excel, Power BI, Python, SQL, Linux, and (later) cloud automation. One artifact, every lane on your roadmap.

The five phases

Phase	What you build	Lane	Time
0. Setup check	Confirm your Debian toolkit	SysAdmin	10 min
1. Excel report	VLOOKUP join + pivot tables (the Bureau workflow)	Analyst	1-2 hrs
2. Pipeline	Python ETL: CSV → clean → SQLite database	Engineer	1 hr
3. SQL practice	12 queries against YOUR database + 3 script mods	Analyst/Eng	ongoing
4. Power BI	Dashboard — the receipt for your resume	Analyst	2-3 hrs
5. GCP lift (Aug)	Automate it in the cloud, all from the CLI	Cloud/SysAdmin	a weekend

The rule: phases in order, one at a time, finish before advancing. A finished Phase 1 beats four started phases. Phases 1–4 are June/July work. Phase 5 is August — do not touch it early; cloud-first with shaky basics builds a project you can't defend in an interview.

Phase 0 — Setup check (Debian terminal)

```
which python3          # path back = installed
which sqlite3          # same
pip list | grep pandas  # need pandas for the pipeline
# Missing something? Install:
pip install pandas --break-system-packages
sudo apt install sqlite3 # (library WiFi blocks apt - do on mobile data/home)
```

Move the project files into your Debian home folder. Check they arrived: `ls ~ | grep -i sfm`

Phase 1 — The Excel Report (Analyst)

Open **SFM_Practice_Data.xlsx**. Two tabs, set up exactly like the real PEARS export: **Program Activity Data** (the lookup: program_id → site_name, borough) and **Sessions** (raw rows — no site names yet, on purpose).

Step 1 — VLOOKUP the site names in

On the Sessions tab, insert a column next to program_id, name it Site Name, and in the first data cell:

```
=VLOOKUP(B2, 'Program Activity Data'!A:C, 2, FALSE)
```

Drag it down the column. This is the exact move from your Bureau guidance doc.

Step 2 — Fix the dates

start_date arrives as text (intentionally). Select the column → Data → Text to Columns → Finish, or format as Date. If a pivot later won't group by month, this step is why.

Step 3 — Build the pivots (the Bureau report)

- **Market Reach:** Insert → PivotTable. Rows: Site Name, then start_date (grouped by month). Values: Sum of num_participants. Filter: is_virtual = 0 (in-person only).
- **Virtual Reach:** copy the pivot to a new tab, flip the filter to is_virtual = 1.
- **Languages:** new pivot — Rows: survey_language. Values: Count of session_id, Sum of num_surveys.
- **EBT/WIC:** Rows: Site Name. Columns: ebt_wic_usage. Values: count.

Finish by writing **3 sentences** on what the data shows (top sites, in-person vs virtual split). That's your 'analyzed the numbers' rep. Phase 1 done = you can demo your Bureau job on demand.

Phase 2 — The Pipeline (Engineer)

In your Debian terminal, with SFM_Practice_Data.csv and load_sfm.py in the same folder:

```
python3 load_sfm.py
```

Expected output: rows extracted → transforms applied → '132 rows in table sessions' → Pipeline complete. That script is a real ETL pipeline: **Extract** (read CSV) → **Transform** (fix dates, add month column, blanks → NULL) → **Load** (write to SQLite). Open the script and read it top to bottom once — every transform is a thing you used to do by hand in Excel, automated.

Phase 2 done = sfm.db exists and you can explain E, T, and L in one sentence each.

Phase 3 — SQL On Your Own Database (Analyst + Engineer)

```
sqlite3 sfm.db
```

Open **practice_queries.sql** and work through all 12 queries — warm-ups, the pivot-table-rebuilt-in-SQL, analyst questions, then two stretch subqueries. **Type them by hand.** No pasting — retyping is where syntax sticks. Query #5 returns your old Excel pivot as a SQL result; that moment is the analyst-to-engineer bridge.

Then the three script modifications (edit `load_sfm.py`, rerun, verify):

- **Easy:** make the final check query also return `SUM(num_participants) AS total_participants`.
- **Medium:** add a transform creating `is_virtual_label` ('Virtual'/'In-Person'). Hint: pandas `.map()` with a dictionary.
- **Stretch:** after loading, run the site-by-month SQL (query #5) from inside Python with `pd.read_sql` and print it.

Phase 3 done = all 12 queries run + all 3 mods work. If you can modify it and predict the result, you own it. This phase also doubles as ongoing train practice — sqlite3 on your phone beats any drill app because it's YOUR data.

Phase 4 — The Power BI Dashboard (Analyst + the receipt)

Power BI Desktop (free, Windows) — library/desktop session. Get Data → CSV → `SFM_Practice_Data.csv`.

- Bar chart: total participants by site (sorted descending)
- Line or column chart: participants by month
- Slicer: `is_virtual` (the interactive version of your old pivot filter)
- Card visuals: total participants, total sessions
- One DAX measure, e.g.: `Total Participants = SUM(sessions[num_participants])`

Screenshot the finished dashboard. **This is the receipt that backs 'Power BI' on your resume** — after this you discuss it in interviews as a thing you built, not a thing you intend to learn.

Publish checkpoint (end of Phase 4): push everything — the script, queries file, dashboard screenshot, and a short README — to GitHub as a portfolio repo. Write-up: what the project is, the workflow it recreates, what each file does. 30 minutes, and the project becomes visible to hiring managers instead of living on your phone.

Phase 5 — The GCP Lift (August, not before)

The same pipeline, automated in the cloud, built **entirely from the gcloud CLI** on your Debian terminal — no web console. That CLI-only detail is the line on the README that impresses hiring managers, because almost nobody at entry level does it.

Architecture (all free tier: BigQuery 10GB storage + 1TB queries/mo, Cloud Storage 5GB — your data is a few hundred KB):

```
# 1. Bucket for the raw CSV (the landing zone)
gcloud storage buckets create gs://YOUR-NAME-sfm-data
gcloud storage cp SFM_Practice_Data.csv gs://YOUR-NAME-sfm-data/

# 2. BigQuery dataset + load (SQLite's big cloud cousin)
bq mk sfm_dataset
bq load --autodetect --source_format=CSV \
  sfm_dataset.sessions gs://YOUR-NAME-sfm-data/SFM_Practice_Data.csv

# 3. Query it - same SQL you practiced in Phase 3
bq query --use_legacy_sql=false \
  'SELECT site_name, SUM(num_participants) AS total
  FROM sfm_dataset.sessions GROUP BY site_name ORDER BY total DESC'

# 4. Automation: Cloud Scheduler (cloud cron) or a cron job on a
#    free e2-micro VM rerunning the load monthly - document either in README
```

Update the GitHub README with the architecture and the exact commands. Phase 5 done = the pipeline runs in the cloud on a schedule, and your repo shows analyst + engineer + sysadmin work in one project.

How to talk about this in interviews

The story (60 seconds): “At the NYC Bureau of Chronic Disease Prevention I worked monthly reporting for a SNAP-Ed nutrition program — pulled program data from the PEARS database, cleaned and joined it in Excel, and built the pivot tables reporting reach across 10–15 farmers market sites and 1,000+ participants. Our 3-person team analyzed the results and presented the season to leadership. Afterward I rebuilt the whole workflow as a portfolio project — first in Excel, then automated it: Python ETL into a SQL database, a Power BI dashboard, and a scheduled cloud version on GCP built from the command line. I can walk you through any layer of it.”

Per-skill receipts: Excel → Phase 1 pivots. SQL → Phase 3, query #5 and the stretch subqueries. Python/ETL → load_sfm.py and your three modifications. Power BI → the Phase 4 dashboard. Linux/CLI → the whole thing ran on your Debian terminal. Cloud → Phase 5. Every claim has a thing you built behind it.

The standard, as always: nothing goes on the resume or out of your mouth that you can't demo or explain. This project exists so that everything can be.

Start with Phase 0 today. Finish phases in order. September is the scoreboard. — built June 12, 2026